DEPARTMENT OF ROBOTICS & ARTIFICIAL INTELLIGENCE

Total Marks: _____

Obtained Marks: _____

Artificial Neural Systems (Lab)

Project Report

Sach Check: Urdu Fake News Detection Using a Feed-Forward Neural Network on WhatsApp

Submission Date: 30th October, 2026

Submitted To:	**Mr. Muhammad Hassan Mujtaba**
Student Name:	**Taimur Sultan**
Registration Number:	**24108145**

Abstract

Fake news written in Urdu spreads quickly through WhatsApp groups, while most automatic fact-checking tools only support English. This project presents **Sach Check**, an artificial neural network system that reads an Urdu news message or news link and estimates the probability that it is fake. The model is trained on the public Ax-to-Grind Urdu benchmark of journalist-labelled news [1]. After removing empty rows, broken labels, a stray header row and 848 duplicate items, 9,235 items remained (4,637 fake and 4,598 real), which were split into 6,464 training (70%), 1,385 validation (15%) and 1,386 unseen test items (15%). Because fake items in this dataset are on average 3.5 times longer than real ones [2], every item was cut to its first 40 words so the model cannot rely on length. Each item was converted into 5,000 TF-IDF features and classified by a feed-forward multi-layer perceptron (5,000 → 128 → 64 → 1) with ReLU, dropout and a sigmoid output, trained with binary cross-entropy and the Adam optimizer. On the held-out test set the network reached an accuracy of **85.9%**, precision of **85.6%**, recall of **86.5%** and an F1-score of **0.861**, outperforming a logistic regression baseline (84.8%, F1 0.846) and a length-only baseline (61.5%, F1 0.476). The trained model was deployed on a Linux server and connected to WhatsApp, where it replies with a probability chart and a short Urdu explanation written by a language model. The report discusses errors, training behaviour, limitations and future directions.

1. Introduction

1.1 Problem Statement

In Pakistan, WhatsApp is one of the main channels through which people receive news. A forwarded message can reach thousands of people within hours, and many of these messages are rumours: fake government announcements, false health cures, or videos shared with the wrong story. Manual fact-checking is slow, and existing tools rarely support Urdu. A system is needed that can read an Urdu message automatically, estimate how likely it is to be fake, and warn the user inside the app they already use, before the message is forwarded again.

1.2 Objectives

- Build an artificial neural network that classifies Urdu news text as real or fake.
- Clean and prepare the Ax-to-Grind Urdu dataset by fixing broken rows, removing duplicates and normalising Urdu letters.
- Control the length shortcut in the dataset by cutting all items to the same length.
- Compare the neural network with a length-only baseline and a logistic regression baseline.
- Evaluate models with a confusion matrix, precision, recall and F1-score, not accuracy alone.
- Deploy the model on a server and make it usable through WhatsApp for both text messages and news links.

1.3 Type of Problem

This is a **supervised binary text classification problem**. For every input news item x, the model predicts one of two classes: **0 = Real** and **1 = Fake**. The network outputs a probability $p \in [0, 1]$ that the item is fake; items with $p \geq 0.5$ are classified as fake.

2. Background

2.1 Artificial Neuron and Feed-Forward Networks

An artificial neuron multiplies each input by a weight, adds a bias term and applies an activation function: output = $f(\Sigma\, w_i x_i + b)$. A **multi-layer perceptron (MLP)**, also called a feed-forward neural network, arranges neurons in layers where information flows in one direction, from the input layer through one or more hidden layers to the output layer. Hidden layers allow the network to learn combinations of input features. In this project the inputs are TF-IDF word scores, so the hidden layers can learn combinations of words that are typical of rumours, which a single-layer linear model such as logistic regression cannot represent.

2.2 Activation Functions

- **ReLU (hidden layers):** $f(x) = \max(0, x)$. It passes positive values and sets negative values to zero, which adds non-linearity and trains efficiently.
- **Sigmoid (output layer):** $f(z) = 1 / (1 + e^{-z})$. It maps the final score to a value between 0 and 1, interpreted as the probability that the news item is fake.

2.3 Loss Function and Optimizer

Binary cross-entropy (BCE) measures how far the predicted probability p is from the true label y, and gives a large penalty when the model is confidently wrong:

$$L_BCE = -[\, y \cdot log(p) + (1 - y) \cdot log(1 - p) \,]$$

The **Adam optimizer** [3] uses backpropagation gradients to update the weights, adapting the step size for each weight. **Dropout** [4] switches off 50% of hidden neurons at random during training, and **early stopping** halts training when validation loss stops improving and restores the best weights. Both techniques reduce overfitting on a small dataset.

2.4 Evaluation Measures

- **Accuracy:** share of all test items classified correctly: (TP + TN) / (TP + TN + FP + FN).
- **Precision (Fake):** of all items predicted as fake, the share that are truly fake: TP / (TP + FP).
- **Recall (Fake):** of all truly fake items, the share the model detects: TP / (TP + FN).
- **F1-Score:** harmonic mean of precision and recall: 2 TP / (2 TP + FP + FN).
- **Confusion Matrix:** counts of true positives, false positives, false negatives and true negatives.
- **Data Leakage Prevention:** duplicate news items were removed before splitting, and the TF-IDF vocabulary was learned from the training set only, so no test information reached the model.

3. Dataset

3.1 Ax-to-Grind Urdu Benchmark

The project uses the public Ax-to-Grind Urdu benchmark (Harris et al. [1]), a collection of Urdu news items from Pakistani and Indian news websites, labelled real or fake by expert journalists and verified through online sources.

Property	Value
Benchmark dataset	Ax-to-Grind Urdu (IEEE TrustCom 2023, public GitHub release)
Raw rows in file	10,106 (10,083 news items + 22 empty rows + 1 repeated header row)
Domains	15: politics, health, sports, economy, technology, religion, showbiz and others
Time period	2017 to 2023
Labels	TRUE (real) and FAKE, encoded as 0 = Real, 1 = Fake
Clean items used	9,235 after removing 848 duplicates and broken rows
Input length	Each item cut to its first 40 words after cleaning
Splitting protocol	Stratified random split, seed 42: 70% train, 15% validation, 15% test

3.2 Data Quality Issues and Fixes

Problem found	Why it matters	Fix applied
22 completely empty rows	They carry no information	Removed
47 labels with a trailing space (“TRUE ”)	Treated as a separate, third label	Spaces stripped
Header row repeated at row 5,053	The file was formed by joining two files	Row removed
848 duplicate news items	Duplicates could appear in both training and test sets	One copy kept
All fake items listed before all real items	An unshuffled split would be biased	Shuffled during split

3.3 Class Distribution

Set	Items	Real	Fake	Fake %	Avg. words (F / R)
Training	6,464 (70%)	3,218	3,246	50.2%	117.0 / 33.1
Validation	1,385 (15%)	690	695	50.2%	115.0 / 32.8
Test	1,386 (15%)	690	696	50.2%	114.6 / 32.5
Total	**9,235 (100%)**	**4,598**	**4,637**	**50.2%**	**116.3 / 33.0**

The dataset is almost perfectly balanced: 50.2% of items are fake and 49.8% are real, and stratified splitting kept this ratio in every set. On the test set, a trivial model that always predicts "real" would reach 49.8% accuracy with a recall of 0.0%, which serves as the lower reference floor.

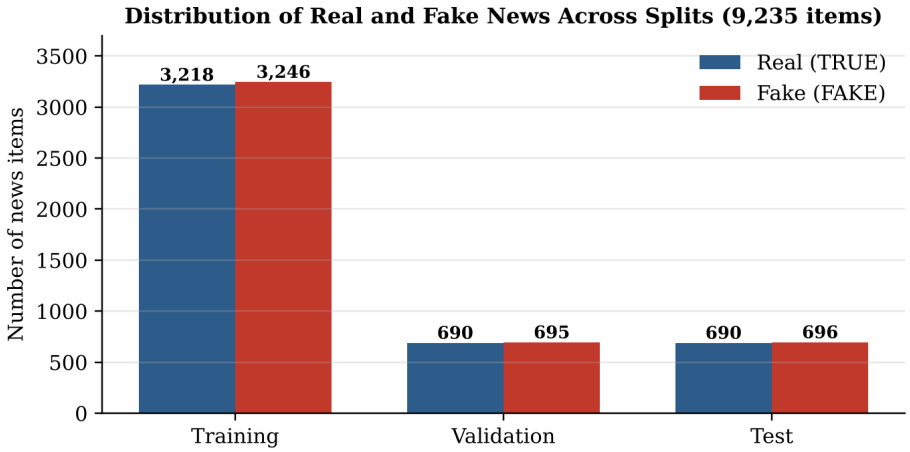


Figure 1: Distribution of real and fake news items across the training, validation and test sets.

A strong length difference exists between the classes, as reported by Haroon [2]: fake items average 116.3 words while real items average 33.0 words. Fake items are a mix of very short rumour headlines and long full articles, while real items are mostly medium-length headlines (Figure 2). To stop the network from learning "long means fake", every item was cut to its first 40 words. This reduced logistic regression accuracy by only about one percentage point, while making inputs closer to the length of real WhatsApp forwards.

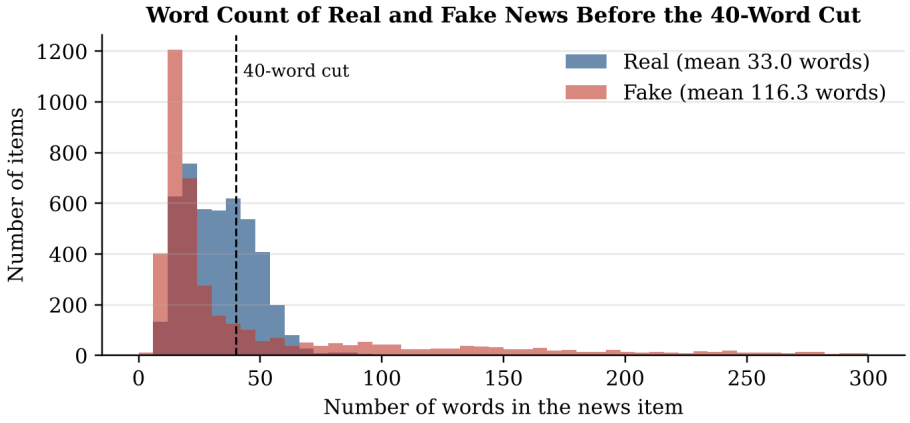


Figure 2: Word count of real and fake items before the 40-word cut (items above 300 words not shown).

4. Methodology

4.1 Pipeline

1. **Data Ingestion:** Load the 10,106 rows of the combined Ax-to-Grind Urdu file.
2. **Data Repair:** Remove empty rows, strip label spaces, drop the repeated header and remove 848 duplicate items.
3. **Text Cleaning:** Apply one cleaning function to every item (Section 4.2) and cut each item to 40 words.
4. **Stratified Splitting:** Split into training (6,464), validation (1,385) and test (1,386) sets with a fixed seed.
5. **Feature Extraction:** Fit a TF-IDF vectorizer (5,000 words) on the training set only and transform all three sets.
6. **Model Training:** Train a length-only baseline, a logistic regression baseline and the feed-forward MLP.
7. **Evaluation and Deployment:** Evaluate all models once on the unseen test set, save the MLP with its vectorizer, and deploy them behind a WhatsApp bot.

4.2 Preprocessing

- **Target Formulation:** Labels were stripped and encoded as binary values: 0 = Real, 1 = Fake.

- **Urdu Normalisation:** Arabic letters that look identical to Urdu letters were replaced by their Urdu forms (for example, Arabic ی became Urdu ی, and Arabic ك became Urdu ک), and diacritics (zer, zabar, pesh) were removed, so the same word is never counted twice.
- **Noise Removal:** Links, English and Urdu digits, punctuation (including ؛ ؟ ۔ ـ) and leading news-agency tags such as "(آن لائن) کوئٹہ" were removed; English words were kept in lowercase.
- **Length Control:** Each cleaned item was cut to its first 40 words; items with fewer than 3 words were dropped (none remained).
- **Vectorisation:** TF-IDF with a vocabulary of the 5,000 most frequent training words turned each item into a 5,000-dimensional vector.

4.3 Model Architectures

Model	Architecture	Trainable params	Type
Length only	Logistic regression on word count	2	Baseline
Logistic regression	Single linear layer + sigmoid on 5,000 TF-IDF inputs	5,001	Baseline
Feed-forward MLP	**Dense 128 (ReLU) → Dropout 0.5 → Dense 64 (ReLU) → Dropout 0.5 → Dense 1 (sigmoid)**	**648,449**	**Proposed**

4.4 Training Settings

Setting	Value
Loss function	Binary cross-entropy
Optimizer	Adam (default learning rate 1×10^{-3})
Batch size and epochs	Batch size 32 (202 steps per epoch), maximum 30 epochs
Regularisation	Dropout 0.5 after each hidden layer
Early stopping	Monitor validation loss, patience 3, restore best weights (best epoch: 2)
Validation split	15% held back (1,385 items), never used for weight updates
Decision threshold	0.5 for evaluation; 0.40 and 0.75 bands for WhatsApp replies (Section 6)
Hardware and tools	Python 3.12, TensorFlow 2.18 / Keras 3, scikit-learn 1.5, pandas, Jupyter, CPU

5. Results

5.1 Model Comparison

Model	Val Accuracy	Test Accuracy	Precision	Recall	F1-Score
Always predict "real"	—	49.8%	—	0.0%	0.000
Length only	61.7%	61.5%	0.750	0.349	0.476
Logistic regression	82.4%	84.8%	0.865	0.828	0.846
Feed-forward MLP	**84.4%**	**85.9%**	**0.856**	**0.865**	**0.861**

Precision, recall and F1-score are reported for the fake class on the 1,386-item test set.



Figure 3: Test accuracy, precision, recall and F1-score across the three models.

Word count alone reaches only **61.5%** accuracy and detects just 34.9% of fake items, so both text models clearly learn from the words rather than the length. The MLP improves test accuracy by **1.1 percentage points** over logistic regression (85.9% vs 84.8%) and raises recall on fake news by **3.7 points** (86.5% vs 82.8%), at the cost of a slightly lower precision (0.856 vs 0.865). The hidden layers therefore help mainly by catching more fake news.

5.2 Confusion Matrices on the Test Set

Evaluated on all 1,386 unseen test items. Rows represent true classes and columns represent model predictions.

Multi-layer network (Feed-forward MLP):

Real \ Predicted	Real	Fake	Total Real	Correct %
Real (690)	589 (TN)	101 (FP)	690	85.36%
Fake (696)	94 (FN)	602 (TP)	696	86.49%

Baseline network (Logistic regression):

Real \ Predicted	Real	Fake	Total Real	Correct %
Real (690)	600 (TN)	90 (FP)	690	86.96%
Fake (696)	120 (FN)	576 (TP)	696	82.76%

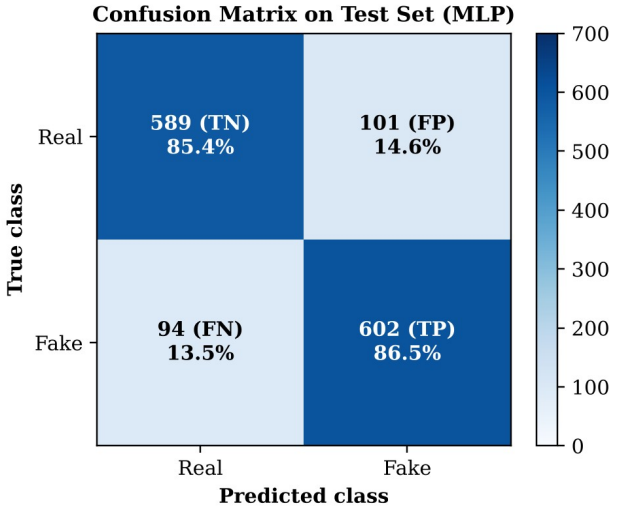


Figure 4: Confusion matrix of the feed-forward MLP on the 1,386-item test set.

The MLP judged **1,191 of 1,386** items correctly. Compared with logistic regression it caught **26 more fake items** (602 against 576) and missed 26 fewer, while producing 11 more false alarms (101 against 90). Its two error types are almost equal (101 false positives, 94 false negatives), so the model does not lean towards either class.

5.3 Precision and Recall of the Multi-Layer Network

Class	Precision	Recall	F1-Score	Support (Items)
Real	0.8624	0.8536	0.8580	690
Fake	0.8563	0.8649	0.8606	696
Overall Accuracy	—	—	**0.8593**	**1,386**

When Sach Check flags an item as fake, it is correct **85.6%** of the time, and it detects **86.5%** of all fake items in the test set. This balance matters for a public tool: too many false alarms would make users stop trusting the bot, while missed fakes would let rumours spread.

5.4 Training Curves

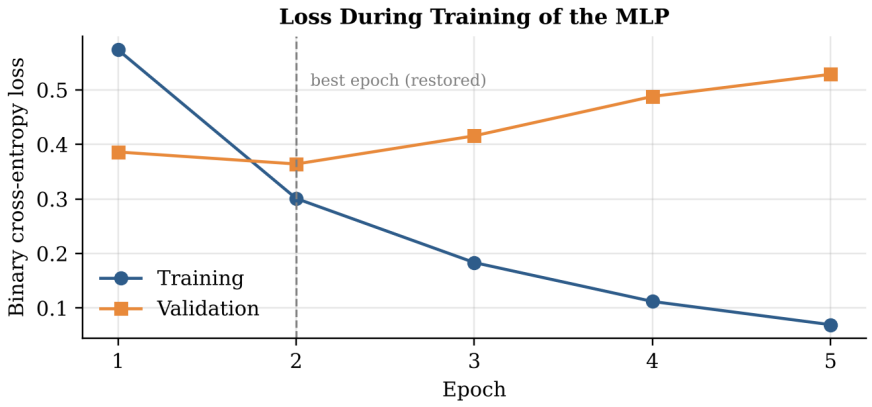


Figure 5: Binary cross-entropy loss during training and validation of the MLP.

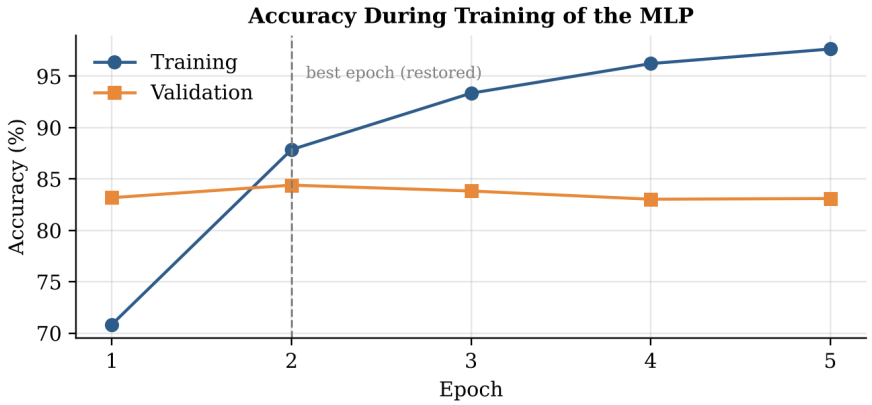


Figure 6: Training and validation accuracy of the MLP across epochs.

Training loss fell steadily from 0.573 to 0.069 over five epochs, and training accuracy rose from 70.8% to 97.7%. Validation loss, however, reached its lowest value of **0.364 at epoch 2** (validation accuracy 84.4%) and then increased to 0.528, while validation accuracy flattened around 83%. This divergence is a clear sign of **overfitting**: with 648,449 weights and only 6,464 training items, the network starts memorising training examples quickly. Early stopping detected the rise in validation loss, stopped training after epoch 5 and restored the weights from epoch 2.

6. System Deployment

The trained network and its TF-IDF vectorizer were saved, pushed to GitHub and pulled onto an Ubuntu Linux server, where two processes run continuously under the pm2 process manager (Figure 7). A self-test on the server reproduced the notebook scores exactly (0.99, 0.19 and 0.97 on three sample messages), confirming that deployment did not change the model.

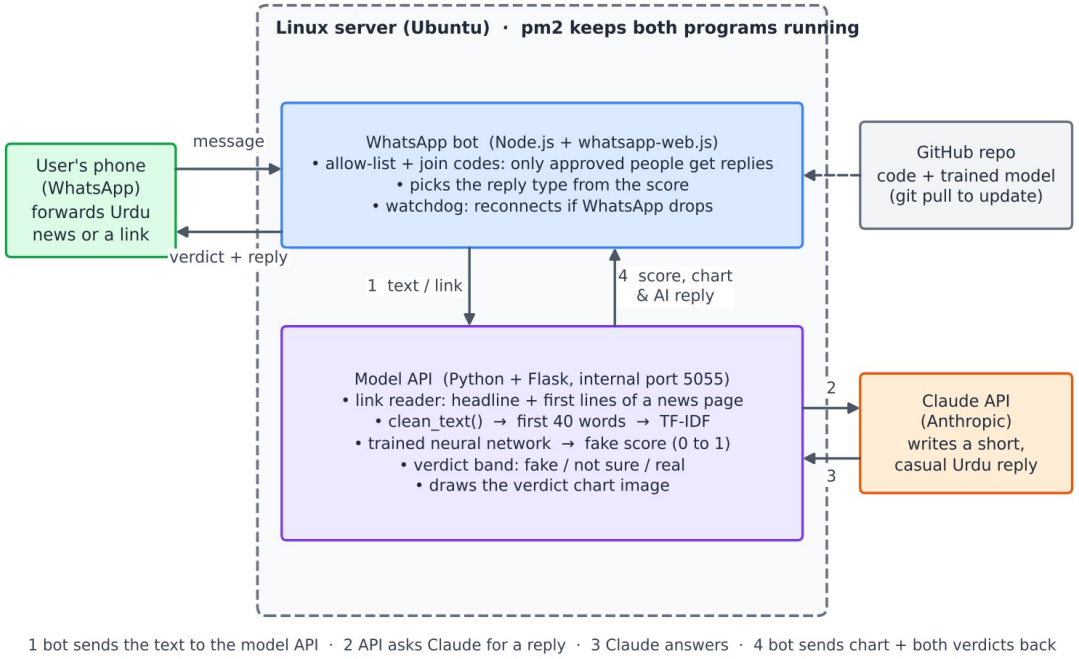


Figure 7: Deployment architecture of the Sach Check system.

- **WhatsApp bot (Node.js, whatsapp-web.js):** receives messages, answers only approved users who join with a code, and reconnects automatically through a watchdog if the WhatsApp connection drops.
- **Model API (Python, Flask):** reads the headline and first lines of a news link, applies the same cleaning and 40-word cut as training, runs the MLP and draws a probability chart.
- **Language model (Claude API):** writes a short, casual Urdu explanation. It is given the network's verdict and is not allowed to change it, and it avoids humour for news about death, accidents or illness.

Fake probability	Verdict	Reply style
$p \geq 0.75$	Likely fake	Light humour and a request not to forward
$0.40 \leq p < 0.75$	Not sure	Neutral, asks the user to verify with a trusted source
$p < 0.40$	Looks like normal news	Calm, but still reminds the user to verify

7. Discussion

Hidden Layers Help Mainly with Recall: The MLP and logistic regression receive the same 5,000 TF-IDF inputs; the only difference is the two hidden layers. These layers allowed the network to learn word combinations, which raised recall on fake news from 82.8% to 86.5%. The overall gain in accuracy is modest (1.1 points), showing that with about 6,500 short texts a linear model already captures much of the signal.

Controlling the Length Shortcut: The length-only baseline reached 61.5%, confirming that length carries real but limited information in this dataset. Cutting every item to 40 words and comparing against this baseline shows that the network's 85.9% comes from word content, not from counting words.

What the Model Learns: The logistic regression weights reveal the strongest signals. Words such as "وائرل" (viral), "ویڈیو" (video), "تصویر" (picture), "دعوی" (claim), "جعلی" (fake) and "کرونا" (corona) push towards fake, matching the language of viral rumours. However, the top pair of opposing words is "لئے" (fake) against "لیے" (real), two spellings of the same word, which shows that part of the signal comes from the writing habits of different source websites rather than from truthfulness.

Overfitting and Regularisation: The network overfits within two epochs despite 50% dropout, because its 648,449 weights greatly outnumber the 6,464 training items. Early stopping was therefore essential; without it, the final model would have been the memorising epoch-5 version with higher validation loss.

From Benchmark to Real Use: Deployment exposed problems that a benchmark score does not show, such as newer WhatsApp versions hiding phone numbers behind internal identifiers, which required a join-code system. Returning the probability with a chart, and always using the word "likely", keeps users aware that the system flags news for checking rather than proving it false.

8. Limitations

- The model judges writing style, not facts; a carefully written false story that imitates official reporting can still be classified as real.
- Messages that warn about a rumour can themselves be flagged as fake, because they contain words such as "جعلی خبر" (fake news).
- Real and fake items come from different websites with different spelling habits, so part of the learned signal reflects the source rather than the content.
- Training data covers news articles from 2017 to 2023; WhatsApp forwards are shorter, more informal and include new topics.
- Only Urdu script is supported; Roman Urdu, English, images, screenshots and voice notes are not checked, and some links (Facebook, YouTube) cannot be opened.
- The current WhatsApp connection uses an unofficial library on a personal number, which is suitable for a prototype but not for large public use.

9. Future Enhancements

Enhancement	Why it could help
Sequence models (LSTM, small transformer)	Read word order and context, which TF-IDF ignores
Real WhatsApp training data	Labelled forwards from volunteers would match real user messages
Roman Urdu and English support	Covers the way many users actually type on WhatsApp
OCR and speech-to-text input	Checks rumours shared as screenshots and voice notes
Fact-check retrieval	Links users to verified articles from trusted sources
Official WhatsApp Business API	Supports many users without the risk of the number being blocked
Message queue, Docker and caching	Scales to many simultaneous checks and reuses results for repeated rumours
Public number and partnerships	Reaches the public through universities, newsrooms and fact-checkers

10. Conclusion

Sach Check, a feed-forward neural network system for Urdu fake news detection, was successfully developed, trained, evaluated and deployed. Careful data repair removed 848 duplicates and broken rows, Urdu letter normalisation made the vocabulary consistent, and a 40-word cut controlled the length shortcut present in the Ax-to-Grind dataset. Using 5,000 TF-IDF features and a two-hidden-layer MLP trained with binary cross-entropy, Adam, dropout and early stopping, the model reached an accuracy of **85.9%**, an F1-score of **0.861** and a fake-news recall of **86.5%** on an unseen test set, outperforming both baselines. Deployed on a Linux server and connected to WhatsApp, the system returns a probability chart and a friendly Urdu explanation within seconds, while the neural network remains the sole decision-maker. Sach Check provides a practical, honest foundation for helping Urdu speakers pause and verify before forwarding news.

References

[1] S. Harris, J. Liu, H. J. Hadi, and Y. Cao, "Ax-to-Grind Urdu: Benchmark Dataset for Urdu Fake News Detection," in Proc. 22nd IEEE Int. Conf. Trust, Security and Privacy in Computing and Communications (TrustCom), 2023. Available: https://github.com/Sheetal83/Ax-to-Grind-Urdu-Dataset

[2] M. A. Haroon, "Cross-Dataset Generalization in Urdu Fake News Detection: An Empirical Study with XLM-RoBERTa and a Length Confound Analysis," arXiv:2607.14131, 2026.

[3] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," in Proc. 3rd ICLR, arXiv:1412.6980, 2015.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[5] M. Amjad, G. Sidorov, A. Zhila, H. Gómez-Adorno, I. Voronkov, and A. Gelbukh, “‘Bend the Truth’: Benchmark Dataset for Fake News Detection in Urdu Language and Its Evaluation,” Journal of Intelligent & Fuzzy Systems, vol. 39, no. 2, pp. 2457–2469, 2020.

[6] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[7] whatsapp-web.js, WhatsApp Web client library for Node.js. Available: https://wwebjs.dev